
Redes neuronales convolucionales (CNN, *Convolutional neural networks*)

Similar a los conceptos abordados en el tema del perceptrón multicapa, las redes neuronales convolucionales (CNN) también presentan múltiples capas que se utilizan para calcular la salida de un conjunto de datos. El desarrollo de este modelo se remonta a la década de 1950, cuando los investigadores Hubel y Wiesel modelaron la corteza visual animal.

En un artículo de 1968, analizaron extensamente sus hallazgos, que **identificaron tanto células simples como complejas en los cerebros de los monos y gatos que estudiaron. Observaron que las células simples presentaban una salida maximizada con respecto a los bordes rectos observados.**

Por el contrario, se observó que el campo receptivo en las células complejas era considerablemente mayor, y sus salidas se veían relativamente poco afectadas por la posición de los bordes dentro de dicho campo. Más allá del reconocimiento de imágenes, por el cual las CNN adquirieron y aún conservan su notoriedad, las CNN tienen otras aplicaciones considerables, como en los campos del procesamiento del lenguaje natural y el aprendizaje por refuerzo.

Estructura y propiedades de las CNN

Las CNN son, en términos generales, **modelos de redes neuronales multicapa**. Siguiendo la estructura de la corteza visual animal descrita por Hubel y Weisel, el modelo puede interpretarse visualmente como se muestra en la figura 1.

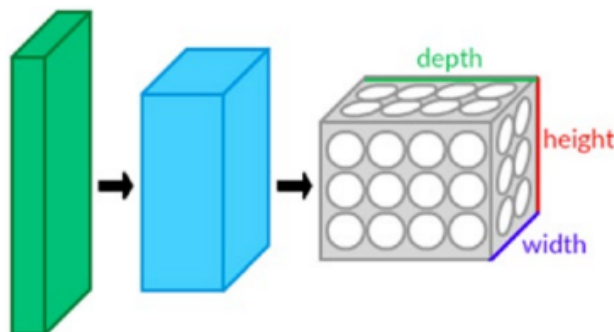


Figura 1: Representación visual general de una CNN.

Cada bloque representa una capa diferente de la CNN, que se explicará con más detalle más adelante en este documento. De izquierda a derecha se encuentran las capas de entrada, oculta

(capas convolucional, de agrupamiento y de abandono – *convolutional, pooling, dropout*) y completamente conectada. Después de la última capa, el modelo genera una clasificación. Consideremos ahora la figura 2.

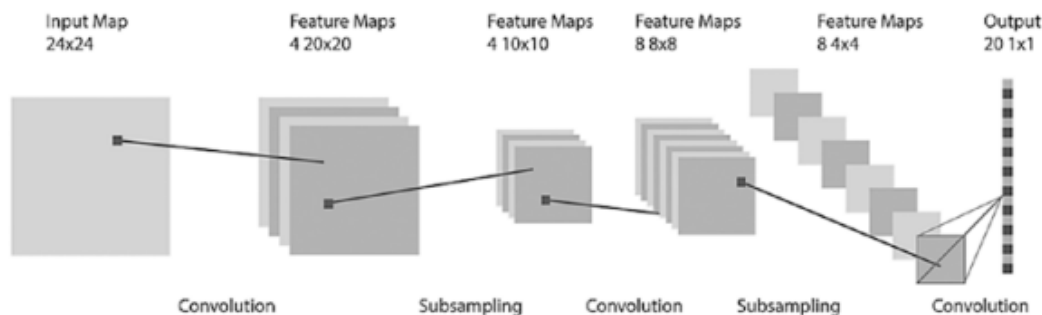


Figura 2: Diagrama de la arquitectura de una CNN.

Las capas completamente conectadas refuerzan la conectividad local entre las neuronas y las capas adyacentes, como se muestra en la figura 2. Por lo tanto, las entradas de las capas ocultas son un subconjunto de neuronas de la capa anterior a dicha capa oculta. Esto garantiza que las neuronas del subconjunto aprendido produzcan la mejor respuesta posible. Además, las unidades comparten el mismo peso y sesgo en el mapa de características/activación, de modo que todas las neuronas de una capa determinada analizan/detectan exactamente la misma característica.

En el contexto de las CNN, las características se refieren a porciones de una imagen que son distintas.

Esto es lo que nuestro filtro compara con la sección de la imagen que analiza, de modo que puede determinar el grado de similitud entre la sección de la imagen escaneada y la característica analizada. Suponiendo que disponemos de suficientes datos de entrenamiento y suficientes clases de imágenes, estas características son lo suficientemente distintas como para ayudar a distinguir una clase de otra.

Imaginemos que observamos dos imágenes, concretamente una X y una O, como en la figura 3.



Figura 3: Ejemplo de foto de O y X.

Si visualizamos la X y la O como imágenes distintas, el ojo humano puede identificarlas como letras claramente diferentes. Entre sus factores distintivos se encuentra que el centro de la O está vacío, mientras que el centro de la X presenta dos líneas que se intersecan. En las figuras 4 y 5 se muestran ejemplos de los mapas de características de estos valores cuando se visualizan.



Figura 4: Mapa de características de “X”.



Figura 5: Mapa de características de “O”.

Estos valores suelen representarse como una entrada dentro de una matriz con -1 y 1 para blanco y negro, respectivamente. Al trabajar con imágenes en color, cada píxel se representa típicamente como una entrada en una matriz con un valor de 1 o 256 para blanco y negro, respectivamente. Sin embargo, dependiendo del lenguaje utilizado, la indexación a cero puede afectar la representación de los valores RGB, de modo que los límites se desplacen hacia atrás en 1.

Componentes de las arquitecturas de CNN

Esta sección abarca los componentes de las arquitecturas de CNN.

Capa convolucional

Esta capa es donde se produce la mayor parte del cálculo en cualquier CNN y, como tal, es la primera capa, después de la entrada, por la que pasa una imagen. Dentro de una capa convolucional, tenemos filtros que escanean una parte de la imagen. No todos los filtros son particularmente grandes en altura y anchura, pero todos se extienden a lo largo de toda la longitud de esta capa.

Por ejemplo, imaginemos que intentamos clasificar una imagen como 1 o 0, y que la imagen en realidad es un 1. Imaginemos que la imagen tiene un fondo negro, pero el dígito está delineado con píxeles blancos. La figura 6 muestra un ejemplo de cómo se ve esta imagen.



Figura 6: Ejemplo de imagen de "1".

La computadora distinguirá los píxeles blancos como 1 y los negros como -1. Al introducir esta imagen a través de la capa convolucional, el modelo extrae sus características únicas, que suelen ser los colores, las formas y los bordes que definen una imagen específica. Una vez que tenemos las características de una imagen de entrenamiento dada, realizamos lo que se conoce como filtrado sobre esta imagen. El *filtrado* consiste en tomar una característica de la imagen, que en este caso podemos imaginar como un cuadrado de 3 x 3 píxeles, y compararla con una porción de esa imagen, que también es un cuadrado de 3 x 3 píxeles. En la figura 7, podemos ver cómo se ve el proceso de filtrado.



Figura 7: Ejemplo de un filtro.

Luego, multiplicamos el número de píxeles de la característica por el número correspondiente de píxeles del parche de imagen. En el ejemplo, deberíamos obtener un resultado de 1 o -1 para cada operación. Intuitivamente, cuando los píxeles coinciden exactamente, deberían tener un resultado de 1, y cuando no, de -1. Finalmente, calculamos el promedio de los productos de píxeles.

Si una imagen coincide exactamente, el promedio debería ser 1. Si no, será considerablemente menor que uno. En este caso, imaginemos que el parche de imagen y la característica seleccionada no coinciden en absoluto. Por lo tanto, al calcular el promedio, el resultado debería ser -1. Colocamos este producto en el centro de la posición del parche de imagen que estamos analizando con una característica dada en lo que se denomina un *mapa de características*.

Este será el resultado de la capa de convolución y se utilizará en la capa siguiente. La capa convolucional, a lo largo de diferentes iteraciones, producirá múltiples mapas de características. El proceso de hacer coincidir una característica con un parche de imagen dado en cada posición posible se conoce como **convolución de una imagen**. Denotamos el mapa de características/activación para una CNN dada como

$$h_{i,j}^k = \tanh((w^k x)_{i,j} + b_k)$$

donde w^k es el peso, b_k es el sesgo, x es el valor del píxel específico y $\tanh$ representa las no linealidades en los datos. Los subíndices i, j se refieren a la entrada de la matriz que representa el mapa de características/activación. El peso, w^k , es en última instancia lo que conecta los píxeles de los mapas de características con la capa anterior. La capa convolucional, en última instancia, es una pila de los mapas de características obtenidos mediante la operación descrita anteriormente. Luego, colocamos los mapas de características en la capa de agrupación. Calculamos el tamaño espacial del volumen de salida como

$$\text{Spatial Size}_{\text{output}} = \frac{W - F + 2P}{S + 1}$$

Donde W = tamaño del volumen de entrada, F = tamaño del campo receptivo de la capa convolucional, P = cantidad de relleno de ceros y S = paso.

Capa de agrupamiento (pooling layer)

Entre capas convolucionales sucesivas, es común colocar una capa de agrupamiento. En pocas palabras, **la capa de agrupamiento toma los mapas de características generados en la capa convolucional y los agrupa en una imagen**. La capa de agrupamiento realiza una **reducción de dimensionalidad** eficaz, de ahí el énfasis en la representación espacial, reduciendo así la complejidad del modelo.

Esto se puede comparar con **el proceso de poda** en árboles de decisión y, de forma similar, ayuda a prevenir el sobreajuste de un modelo dado. En el modelo CNN prototípico, la capa de agrupamiento tiene un filtro 2 x 2, un paso de 2 y cada corte de profundidad en la entrada se submuestra de forma que se desplaza 2 píxeles con respecto a la altura y el ancho. Estas operaciones en la capa de agrupamiento ayudan a descartar el 75 % del mapa de características/activación. Esta capa utiliza una operación de máx., que en el ejemplo mencionado tomaría un máximo de 4 números, o los 4 píxeles de cualquier mapa de características/activación.

Siguiendo el ejemplo descrito anteriormente, imaginemos que con un filtro 2 x 2, observamos un mapa de características de 9 x 9, donde analizamos la esquina superior izquierda con las siguientes puntuaciones:

.88	0
0	.95

Al usar la operación de máx., elegiríamos 0.95, ya que es el valor máximo dentro de la ventana 2 x 2. Dado que tenemos un paso de 2, nos desplazamos 2 píxeles a la derecha, lo que significa que observamos un segmento 2 x 2 de la imagen, donde la esquina superior izquierda del segmento debería ser la tercera columna del mapa de características hasta obtener una imagen agrupada máxima, lo cual se reduce significativamente y, por lo tanto, elimina complejidades innecesarias del modelo.

Como consecuencia directa de la operación máxima utilizada en esta capa, no necesitamos ser tan precisos como en la capa anterior al analizar la imagen, lo que ayuda a crear un modelo más robusto que permite clasificar las entradas con mayor facilidad. Con esto quiero decir específicamente que los valores de los pesos que conectan cada capa pueden generalizarse mejor a todos los datos de entrenamiento a los que se han expuesto, en lugar de sobreajustarse de tal manera que la CNN no tendría un buen rendimiento fuera de la muestra.

La función que determina el tamaño espacial de la salida viene dada por

$$\text{Spatial Size}_{\text{output}} = W_2 \times H_2 \times L_2,$$

donde

$$W_2 = \frac{w_1 - F}{s + 1}, H_2 = \frac{H_1 - F}{S + 1}, L_2 = L_1$$

Capa de unidades lineales rectificadas (ReLU)

Los *rectificadores* se utilizan como otro término para una función de activación. Normalmente, aplicamos la siguiente función a las entradas de esta capa:

$$f(x) = \max(0, x)$$

donde x = entrada a una neurona.

Al aplicarlo al mapa de características, podemos imaginar que cualquiera de los valores del mapa de características que sería negativo ahora es cero. En concreto, esto ayuda a delimitar el mapa de características más cerca de la imagen con la que está más asociado. Hacemos esto con todos los mapas de características para obtener una “pila” de imágenes.

Capa completamente conectada (FC, *Fully connected*)

Todas las neuronas de esta capa están conectadas a todos los mapas de activación de la capa anterior.

Esta capa suele colocarse después de una cantidad determinada por el usuario de capas convolucionales, de agrupamiento y ReLU. Las imágenes introducidas en esta capa serán significativamente más pequeñas que las entradas originales debido a las reducciones de imagen especificadas en las operaciones anteriores. En esta capa, escaneamos las imágenes reducidas, que deberían corresponder a cada mapa de características, y convertimos cada uno de los valores dados aquí en una lista de valores. Esta lista corresponde a una de las k imágenes introducidas.

Siguiendo el ejemplo utilizado al principio del documento, inicialmente introducimos un 1. Tras recorrer todas las capas, calculamos el promedio de las puntuaciones correspondientes a esta imagen, que es la probabilidad de que la imagen sea un 1 o un 0. Cabe destacar que la única diferencia entre esta capa y la capa convolucional es que las capas convolucionales solo están conectadas a una región local en la entrada y que muchas de las neuronas en una capa convolucional comparten parámetros de volumen. Con esto en mente, también podemos convertir entre capas FC y convolucionales al construir una arquitectura determinada.

Capa de pérdida (Loss layer)

En esta capa comparamos las etiquetas predichas con las etiquetas reales de las imágenes. Al clasificar un objeto a partir de k niveles de características posibles, utilizamos un clasificador de pérdida softmax. También es común usar una función euclidiana para realizar regresión con las etiquetas de las imágenes específicas. Sus funciones se definen de la siguiente manera:

I. Función de pérdida softmax:

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_{k=1}^n e^{z_k}}$$

II. Función de pérdida euclidiana:

$$E = \frac{1}{2N} \sum_{i=1}^N \|\hat{y}_i - y_i\|_2^2$$

III. Normalización softmax:

$$x'_i = \frac{1}{1 + e^{-\left(\frac{x_i - \mu_i}{\sigma_i}\right)}}$$

Al utilizar el algoritmo de retropropagación, creamos una matriz de confusión que compara un 1 con un 0, donde restamos la etiqueta de la respuesta a la probabilidad asignada. Siguiendo el ejemplo que hemos estado utilizando, supongamos que $1 = 1$ y $0 = 0$, pero al introducir un 1, solo recibimos una probabilidad de 0.85 de que sea 0 y una probabilidad de 0.45 de que sea 1.

Por lo tanto, tendríamos un error acumulado de -0.60. A continuación, **ajustamos cada píxel del mapa de características mediante los pesos mostrados** en la capa completamente conectada, **utilizando un método de gradiente** como el descrito anteriormente, con una tasa de aprendizaje designada. Inicializamos los pesos en 0 y detenemos la CNN cuando se alcanza la tolerancia a la pérdida o el umbral máximo de iteraciones. Para la convergencia a una solución óptima, se deben tener en cuenta las mismas consideraciones descritas en los temas de perceptrones.

Parámetros de ajuste

Las imágenes enviadas a la capa de entrada deben ser divisibles por 2 más de una vez. Las **dimensiones comunes de las imágenes** son 32×32 , 64×64 , etc. Las capas convolucionales deben tener filtros con dimensiones de 3×3 o 5×5 como máximo, y el relleno de ceros debe realizarse de forma que no altere las dimensiones espaciales de la entrada. Para las capas de agrupamiento, sus dimensiones deben ser 2×2 , con un paso de 2 en la mayoría de los casos. Con estos parámetros, se descartará el 75 % de las activaciones. La agrupación de capas mayores de 3 genera una pérdida excesiva en el proceso de clasificación. Al describir las neuronas y sus disposiciones, los hiperparámetros son los más relevantes en este contexto.

Específicamente, nos referimos a la **zancada (*stride*)**, la **profundidad (*depth*)** y el **relleno de ceros (*zero-padding*)**. Entre los parámetros más importantes en las CNN, la **zancada** es un parámetro fijo que **determina el número de píxeles que se deslizan a través de un filtro**. Por ejemplo, si la zancada es 2, entonces se deslizan 2 píxeles a la vez a través de los filtros. Normalmente, la zancada no es mayor que 2 ni menor que 1. El **relleno de ceros es el tamaño de los ceros alrededor del borde del volumen de entrada**. Al controlar el relleno de ceros, podemos controlar con mayor precisión el tamaño de los mapas de activación y otras salidas, de una capa a otra. Finalmente, **la profundidad se refiere al número de filtros que elegimos para un experimento dado**, cada uno de los cuales es lo que finalmente busca en cada imagen en la capa convolucional.

Arquitecturas CNN destacadas

LeNet: Desarrollada en la década de 1990 por el reconocido investigador de aprendizaje profundo Yann LeCun, LeNet es una arquitectura relativamente simple. El propósito original de este modelo era clasificar dígitos, leer códigos postales y realizar una clasificación general

de imágenes. Se considera el análogo de un programa “Hola Mundo” que cualquier desarrollador escribe primero en un lenguaje determinado, ya que se considera la primera aplicación exitosa de CNN en una tarea práctica. Como ilustra la figura 8, las capas involucradas son las siguientes:

- entrada, capa de convolución, ReLU, capa de agrupación, capa de convolución, ReLU, capa de agrupación, completamente conectada, ReLU, completamente conectada y clasificador softmax.

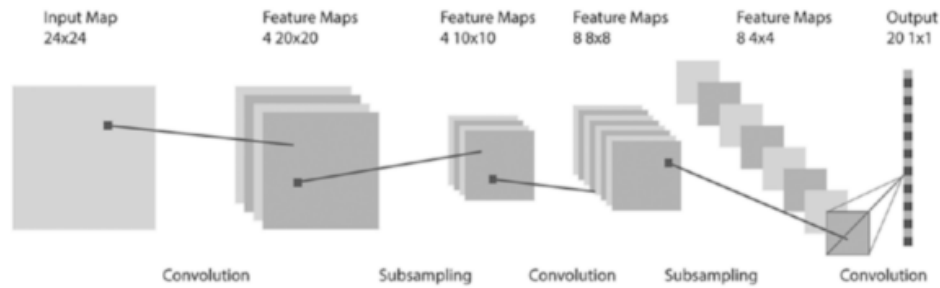


Figura 8: Visualización de LeNet.

GoogLeNet (Inception): Esta arquitectura ganó el concurso ImageNet Large-Scale Visual Recognition Challenge (ILSVRC) en 2014 en homenaje a LeNet de Yann LeCun. Fue desarrollada por Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Cinven Vanhoucke y Andrew Rabinovich. El nombre GoogLeNet se debe a que un número considerable de los desarrolladores de la arquitectura trabajan en Google Inc. En su artículo “Going Deeper with Convolutions”, describen una arquitectura que permite “aumentar la profundidad y la anchura de la red manteniendo la restricción computacional”. Como ilustra la figura 9, la estructura propuesta es la siguiente:

- Entrada, conv.layer, grupo máximo, capa de convolución, capa de agrupación (con función max), inception (2 capas), grupo máximo, inception, inception (5 capas), grupo máximo, inception (2 capas), capa de agrupación promedio, abandono, ReLU, clasificador softmax.

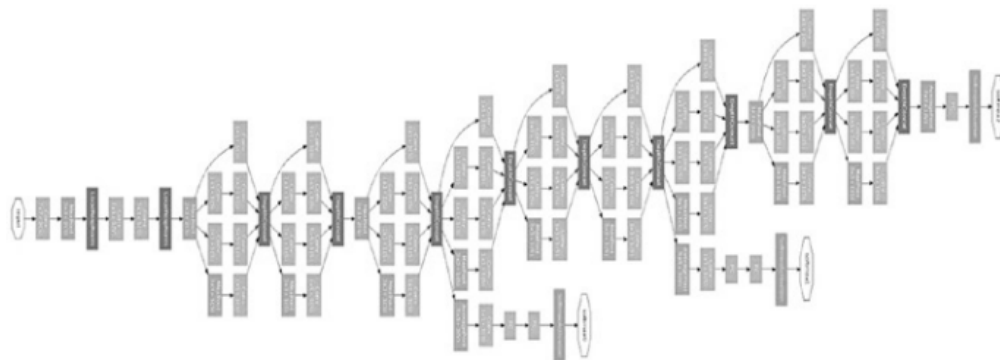


Figura 9: Arquitectura de GoogLeNet.

El enfoque de la arquitectura de inception radica en que, mediante la orientación en las capas descrita anteriormente, el modelo CNN permite aumentar el número de unidades en cada etapa sin llegar al punto de que el modelo se vuelva demasiado complejo. En general, el modelo busca procesar información visual a diversas escalas y luego agregar los cálculos a la siguiente etapa para que se analicen simultáneamente niveles de abstracción más altos.

AlexNet: Desarrollada por Alex Krizhevsky, Ilya Sutskever y Geoffrey Hinton, ganó el ILSVRC en 2012. Similar en arquitectura a LeNet, AlexNet utiliza neuronas no saturadas e implementa eficientemente la GPU para las capas de convolución. Las neuronas en las capas completamente conectadas se conectan a todas las neuronas de la capa anterior; las capas de normalización de respuesta siguen a la primera y la segunda capa convolucional, y los núcleos de las capas dos, cuatro y cinco se conectan únicamente a los mapas de núcleos de la capa anterior, que estarían en la misma GPU. La arquitectura es la siguiente (mostrada en la figura 10):

- Convolucional (5 capas), completamente conectada (3 capas), [la salida es un clasificador softmax de 1000 vías]

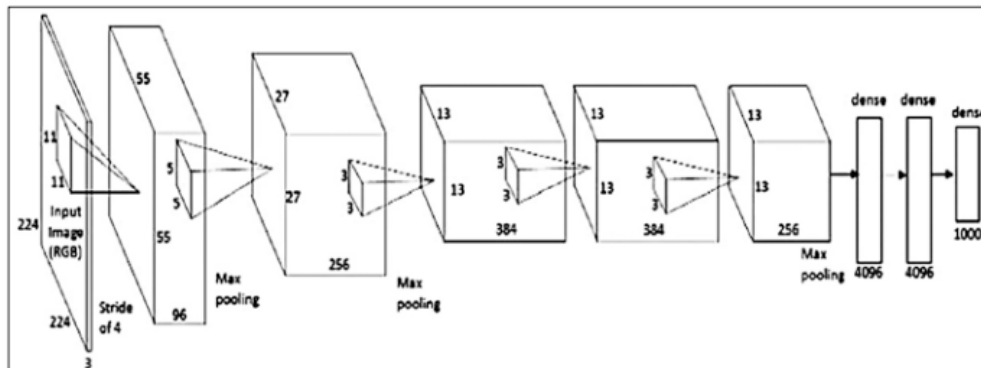


Figura 10: Arquitectura de AlexNet.

VGGNet: Obtuvo el segundo puesto, superado por AlexNet, en la competencia ILSVRC 2014. VGGNet fue desarrollada por Karen Simonyan y Andrew Zisserman de la Universidad de Oxford. El campo receptivo es de 3 x 3, con filtros de 1 x 1, un paso de 2 y un tamaño máximo de agrupación de 2 x 2. La arquitectura permite que la entrada se distribuya a través de varias capas convolucionales hasta llegar a tres capas completamente conectadas (la primera y la segunda capa tienen 4096 canales, y la última es una capa softmax que realiza una clasificación de 1000 vías).

ResNet: Ganadora del primer puesto en ILSVRC 2015, ResNet cuenta con 152 capas, superando con creces la cantidad de las redes mencionadas anteriormente. Fue desarrollada por Kaimin He, Xiangyu Zhang, Shaoqing Ren y Jian Sun, todos de Microsoft Research. El objetivo de esta arquitectura es formar una red que aprenda funciones residuales con

referencias a las entradas de las capas, en lugar de una red que aprenda funciones sin referencia. El resultado final es una red considerablemente más fácil de aprender, significativamente más fácil de optimizar y que gana precisión a medida que aumenta la profundidad, en lugar de una que la pierde con esa profundidad.

Regularización

Cuando los perceptrones multicapa tienen más de una capa, se sabe que tienen la capacidad de aproximarse a un objetivo determinado, lo que podría provocar sobreajuste. Para evitarlo, se suele recomendar regularizar los datos de entrada; sin embargo, este proceso es ligeramente diferente en el caso de las CNN. Podemos utilizar: 1) **DropOut**, que se inspira en un fenómeno observado en el cerebro humano. En este caso, una capa oculta dada tiene la probabilidad de no ser superada con la probabilidad establecida como hiperparámetro. 2) **Agrupación estocástica**, donde la activación se selecciona aleatoriamente.

La agrupación estocástica no requiere hiperparámetros y puede utilizarse como una heurística, por así decirlo, con otras técnicas de regularización. 3) **DropConnect**, que es una generalización de dropout, donde cada conexión puede descartarse con una probabilidad de $1 - p$. Cada unidad de esta capa introduce datos de unidades aleatorias de la capa anterior, que cambian en cada iteración. Esto ayuda a garantizar que los pesos no se sobreajusten. 4) **Decaimiento de Peso**, que funciona de forma similar a la regularización L1/L2, donde penalizamos considerablemente los vectores de peso grandes.

De estos métodos, el uso de DropOut en CNN ha generado un gran entusiasmo, ya que ha demostrado ser una técnica eficaz y potente. Además de prevenir el sobreajuste, se ha observado que DropOut mejora la eficiencia computacional de redes con grandes cantidades de parámetros, ya que esta forma de regularización hace que la red se reduzca durante una iteración determinada. Tras todas estas iteraciones, el rendimiento de las redes más pequeñas puede promediarse para obtener una predicción general del rendimiento de una red completa. En segundo lugar, se observa que la capa DropOut introduce un rendimiento aleatorio en la red que permite promediar el ruido en los datos, de modo que se reduce el enmascaramiento de las señales.

No es raro utilizar la regularización L1, pero ten en cuenta que, en este caso, los vectores de peso a menudo pueden reducirse a cero, a veces lo suficiente como para que quede una matriz de peso escasamente poblada. El efecto negativo de este tipo de regularización es que las entradas a ciertas capas que contienen información importante pueden pasar completamente desapercibidas debido a una conexión “inactiva” entre capas. Sin embargo, cuando se desea específicamente una selección de características muy explícita, la regularización L1 puede ofrecer un rendimiento significativamente mejor.

La regularización L2 se considera tradicionalmente el método estándar de regularización en CNN, ya que tiende a penalizar pesos anormalmente grandes y favorecer aquellos con una proporción generalmente baja en relación con la totalidad de la matriz. A diferencia de la regularización L1, se obtiene una matriz de pesos considerablemente más poblada, lo que hará que la red transfiera más datos de una capa a la siguiente. Por lo tanto, la selección de características será menos estricta que con la regularización L1.

El último tipo de regularización que debe conocer sería una adición a la regularización L1 o L2 mediante la imposición de límites al tamaño de la norma de un peso determinado. Por lo tanto, esto permitiría que las actualizaciones de parámetros tuvieran un límite estricto y, por lo tanto, limitaría el número de posibles soluciones que una red puede generar. Esto ayudaría a entrenar la red más rápido a través de la limitación de posibles soluciones y, en la solución óptima, evitaría que los parámetros se actualicen demasiado en la dirección incorrecta.

Resumen

Hemos cubierto suficientemente los conceptos de las CNN y hemos repasado todas las arquitecturas más recientes. Veremos algunos ejemplos para entender más su aplicación.